Feedforward Neural Network for CIFAR-10 Image Classification: MLP Training with Backpropagation and Comparison to k-NN and NCC

Dimitrios Netskas
Undergraduate student at the Department of Informatics, Aristotle University of Thessaloniki
AEM:4341

***Abstract*—This report presents the implementation and evaluation of a fully connected feedforward neural network (Multi-Layer Perceptron, MLP) trained with the back-propagation algorithm on the CIFAR-10 image classification task. The network is implemented from scratch (without deep learning frameworks) and is trained in a supervised way on raw image pixels after appropriate normalization. The impact of different architectural and training hyperparameters is investigated, such as the number of hidden neurons and hidden layers, the learning rate and the use of mini-batch versus full-batch gradient descent. The performance of the MLP is compared against two classical baseline classifiers: the k-Nearest Neighbor (with $k = 1$ and $k = 3$) and the Nearest Class Centroid that were implemented in an earlier assignment also from scratch. Results are reported in terms of training and test accuracy, loss curves, training time and example correct and incorrect classifications, followed by a discussion of advantages and limitations of the MLP approach on CIFAR-10.**

I. Introduction

DEEP learning methods, and in particular artificial neural networks, have become the dominant approach for solving large-scale classification problems. In this project we focus on a simple yet fundamental architecture, the fully connected feedforward neural network or Multi-Layer Perceptron (MLP), and apply it to the well-known CIFAR-10 image classification dataset.

The goals of the assignment are three-fold: (i) to implement from scratch a feedforward neural network trained with the back-propagation algorithm, (ii) to study how architectural and training hyperparameters influence the final performance in the pursuit of maximize the model's accuracy, and (iii) to compare the obtained results to classical pattern recognition methods, specifically the k-Nearest Neighbor (k-NN) classifier and the Nearest Class Centroid (NCC), which were implemented in the intermediate assignment.

The remainder of this report is organized as follows. Section II describes the CIFAR-10 dataset and the preprocessing steps. Section III presents the architecture of the MLP and the baseline classifiers. The experimental setup and training procedure are detailed in Section IV. Section V reports and analyzes the experimental results, including examples of correct and incorrect classifications. Section VII demonstrates the code analysis and how to reproduce the final experimen. Section VIII provides a discussion of the observed behavior of the models, and Section IX summarizes the main conclusions.

II. Dataset and Preprocessing

A. CIFAR-10 Dataset

The CIFAR-10 dataset consists of 60,000 color images of size 32×32 pixels with three color channels (RGB), divided into 10 object classes (airplanes, cars, birds, cats, deer, dogs, frogs, horses, ships, and trucks). We devide those 60,000 into 50,000 images in the training set and 10,000 images in the test set. Each image is associated with a single class label.

B. Input Representation

Each image is reshaped from its original $32 \times 32 \times 3$ tensor into a one-dimensional vector of 3,072 features by simple concatenation of all pixel values. Thus, the input layer of the MLP has 3,072 neurons.

C. Normalization

To facilitate stable training, per-channel normalization is applied using statistics computed on the training set. For each color channel we subtract the mean and divide by the standard deviation of that channel over all training images. The same transformation is then applied to the test set.

III. Methods

A. Multi-Layer Perceptron (MLP)

The main model used in this work is a fully connected MLP with two hidden layers. The architecture is as follows:

- Input layer: 3,072 units (normalized pixel intensities).
- First hidden layer: $H1$ units
- Second hidden layer: $H2$ units
- Output layer: 10 units (one per given classes) with softmax.

For an input vector $\mathbf{x} \in \mathbb{R}^{3072}$, the forward pass is given by

$$\mathbf{h}_1 = \phi(W_1\mathbf{x} + \mathbf{b}_1), \tag{1}$$

$$\mathbf{h}_2 = \phi(W_2\mathbf{h}_1 + \mathbf{b}_2), \tag{2}$$

$$\mathbf{o} = W_3\mathbf{h}_2 + \mathbf{b}_3, \tag{3}$$

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{o}), \tag{4}$$

where $W_1 \in \mathbb{R}^{192\times 3072}$, $W_2 \in \mathbb{R}^{96\times 192}$, and $W_3 \in \mathbb{R}^{10\times 96}$ are the weight matrices, $\mathbf{b}_i$ are the bias vectors, and $\phi(\cdot) = \max(0, \cdot)$ is the **Rectified Linear Unit (ReLU)** activation.

1) Loss Function: The network is trained by minimizing the categorical cross-entropy loss $\mathcal{L}_{\text{CE}}$ (with **Label Smoothing** $\epsilon = 0.05$):

$$\mathcal{L}_{\text{CE}}(\mathbf{y}, \hat{\mathbf{y}}) = -\sum_{c=1}^{10} y_c \log \hat{y}_c. \tag{5}$$

In all optimized experiments, an L2 regularization term is added to the loss. The training procedure also incorporates **Momentum** and **Dropout** in the hidden layers.

2) Weight Initialization: Weights of the first and second layer are initialized using He initialization appropriate for ReLU activations. Biases are initialized to zero.

B. Training with Back-Propagation

The network is trained using stochastic gradient descent (SGD) with the back-propagation algorithm. At each iteration, a mini-batch of training examples is forward propagated through the network, the loss is computed, and gradients with respect to all parameters are obtained via back-propagation. Parameters are then updated as

$$\theta \leftarrow \theta - \alpha \, \nabla_\theta \mathcal{L}_{\text{batch}}, \tag{6}$$

where α is the learning rate and θ denotes collectively all trainable parameters (W_1, b_1, W_2, b_2, W_3, b_3).

The implementation is done in a numerical computing environment (NumPy) without the use of high-level deep learning frameworks, in order to explicitly study and understand the forward and backward passes of the MLP and what is the limit of this model.

C. Baseline Classifiers

For comparison purposes, two classical classifiers are also evaluated on the same dataset and feature representation:

1) k-Nearest Neighbor (k-NN): The k-NN classifier assigns to each test sample the majority label among its k closest training samples using Euclidean distance in the input space. In this report $k = 1$ and $k = 3$ have been examined.

2) Nearest Class Centroid (NCC): The NCC classifier computes for each class the centroid (mean vector) of its training samples. A test sample is then assigned to the class whose centroid is closest in Euclidean distance to the sample.

These two methods were implemented in the intermediate assignment and are reexamined here to provide a baseline comparison for the performance of the MLP.

IV. Experimental Setup

A. Hyperparameters

Different values are explored in Section V to study their effect on generalization performance. In pursuit of maximizing the model's accuracy.

B. Train-Test Splits

The original CIFAR-10 training set is split into a training set of 5,0000 and test set of 10,000 images.

C. Implementation Details

All training experiments are conducted in Python using the Google Colab environment, on a single CPU instance with 12 GB of RAM and 108 GB of available disk space. The code logs the training and test loss and accuracy at the end of each epoch into a metrics.csv file, which is later used to generate learning–curve plots. In addition, each run records its configuration, including the main hyperparameters (e.g., learning rate, batch size, optimization settings), so that experiments are fully reproducible. For every experiment, all outputs are saved in a Google Drive folder named `nn_experiments`, allowing the results of different runs to be examined and compared after any change in the model or training setup.

V. Results

A. MLP Performance and Experimental Evaluation

A systematic series of experiments was conducted to configure and optimize the final Multi-Layer Perceptron (MLP) model. Each configuration was rigorously evaluated, focusing on the critical trade-off between **model capacity** and **generalization performance** to effectively mitigate the effects of overfitting and underfitting. The hyperparameter space explored included: the **learning rate schedule**, **batch size**, number of **hidden layers and units**, choice of **activation function**, influence of various **regularization techniques** (L2, Dropout), and the **optimizer**. The results, summarized in the subsequent tables and figures, report the attained training and test accuracy, the **generalization gap** (the difference between training and test accuracy), as well as the total training time, detailing the procedure followed to achieve the optimal balance between model complexity and predictive power.

B. Experiments

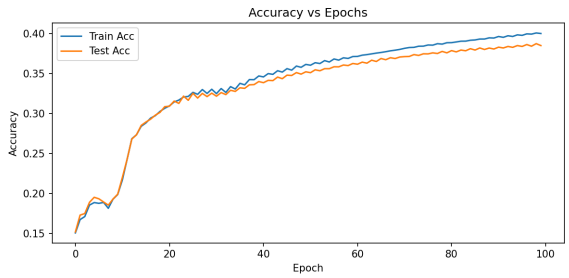
*1) **Experiment 1**: Initial Baseline Performance:* The initial experiment established a baseline using a minimalistic architecture (Table I). The performance of the model, detailed in Table II, was poor ($\approx \mathbf{40}\%$ accuracy). The model lacked the capacity (only 32 hidden units) and the training strength to learn the complexity of the CIFAR-10 data. The low generalization gap confirms that the low accuracy was due to the model failing to fit the training data. This observation mandated an immediate increase in model complexity.

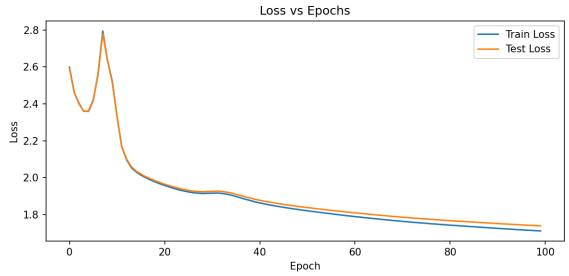
TABLE I
EXPERIMENT 1 CONFIGURATION: INITIAL BASELINE MODEL

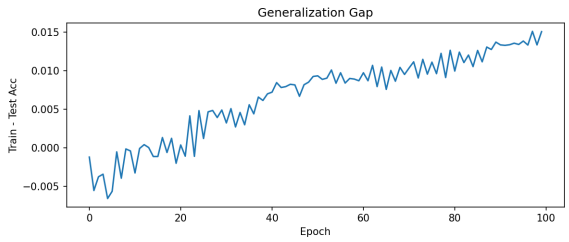
Parameter	Value
Architecture	$3072 \to 32 \to 10$
Hidden Layers	1
Units (H1)	32
Learning rate (α)	0.03
Optimizer	SGD
Batch Size	Full-batch
Activation	ReLU
Normalization	Per-channel Z-score
Epochs	100

TABLE II
EXPERIMENT 1 RESULTS (EPOCH 100)

Metric	Value
Training accuracy (%)	39.98
Test accuracy (%)	38.48
Training loss	1.71
Test loss	1.73
Generalization Gap (%)	1.50
Training time (min)	24







*2) **Experiment 2**: Focusing on Model Capacity:* Following the low accuracy observed in Experiment 1, Experiment 2 focused on increasing the single hidden layer's size to determine how model **capacity** affects performance. Also He initialization was introduced as it is believed to be one of the best initialization for the ReLU activation function. As shown in Table IV, the other training parameters ($\alpha = 0.03$, full-batch) were kept constant to isolate the effect of the number of hidden units (H). The configurations explored were $H \in \{128, 256, 512\}$.

The results in Table IV show a clear trend: as model capacity was increased, both the **Training Accuracy** and the **Test Accuracy** improved (reaching a maximum of 44.32% and 42.88% with $H = 512$). This confirms that the model was successfully moved out of the limit it had reached with 32 units in the first Experiment and reveals that furthermore more units must be used in the following experiments, in order to have better results.

TABLE III
EXPERIMENT 2 CONFIGURATION: DIFFENT CAPACITIES

Parameter	**Value**
Architecture	$3072 \to H \to 10$
Hidden Layers	1
Units	128, 256, 512
Learning rate (α)	0.03
Optimizer	SGD
Batch Size	Full-batch
Activation	ReLU
Initialization	Kaiming He initialization
Normalization	Per-channel Z-score
Epochs	100

TABLE IV
EXPERIMENT 2 RESULTS (EPOCH 100)

Metric	Value (H=128)	Value (H=256)	Value (H=512)
Training accuracy (%)	42.84	43.68	44.32
Test accuracy (%)	41.53	41.65	42.88
Training loss	1.65	1.62	1.59
Test loss	1.67	1.65	1.64
Generalization Gap (%)	1.31	2.03	1.52
Training time (min)	30	35	41

However, we observed that more time must be spent on training while we increase the number of units. Also, the generalization gap stayed relevantly close to Experiment 1, a clue that indicates that we are on the right path.

*3) **Experiment 3**: Introducing Mini-Batches and Learning Rate Tuning:* Experiment 1 highlighted low accuracy, while Experiment 2 confirmed that using a large model capacity (e.g., $H = 256$) with the high initial learning rate ($\alpha = 0.03$) and full-batch resulted in higher accuracy. In order to have a better accuracy and try to minimize both training and test loss these changes were implemented:

1) Transition from full-batch to Mini-Batch Gradient Descent (Batch Size $B = 256$) to stabilize training.
2) Systematic tuning of the Learning Rate (α) across different model capacities.
3) Also the application of a different activation function (Leaky ReLU) was applied to some of the following experiments.

TABLE V
EXPERIMENT 3 CONFIGURATION CHANGES (VS. EXPERIMENT 1)

Parameter	**Experiment 1**	**Experiment 3 Base**
Architecture Base	$3072 \to 32 \to 10$	$3072 \to H \to 10$
Optimizer	SGD (Full-batch)	SGD (Mini-Batch 256)
Hidden Units	32	32, 256, 512
Learning Rate (α)	0.03	0.03, 0.01, 0.003, 0.001
Initialization	-	Kaiming He initialization

The results in Table VII clearly demonstrate the critical role of the learning rate:

- High α (0.03, 0.01) : This resulted in catastrophic overfitting Gap (both for H=32 and H=256), indicating that the model was learning too aggressively from mini-batches and diverging from the optimum. This led the model to extreme overfitting.
- Reducing α: Progressively reducing the learning rate (from 0.03 to 0.001) drastically reduced the generaliza-

TABLE VI
EXPERIMENT 3 RESULTS: TUNING CAPACITY (H) AND LEARNING RATE (α)

Config	Tr.Acc	Te.Acc	Tr.Loss	Te.Loss	Gap
$H = 32, \alpha = 0.03$	61.22	45.57	1.09	1.67	15.65
$H = 256, \alpha = 0.03$	92.74	49.23	0.25	2.39	43.51
$H = 256, \alpha = 0.01$	85.54	50.45	0.50	1.62	35.09
$H = 256, \alpha = 0.001$	56.04	49.14	1.29	1.45	6.90
$H = 512, \alpha = 0.001$	57.53	49.85	1.25	1.43	7.68
$H = 512, \alpha = 0.003$	70.30	53.09	0.92	1.37	17.21

TABLE VII
EXPERIMENT 3 RESULTS: LEAKY RELU

Config	Tr.Acc	Te.Acc	Tr.Loss	Te.Loss	Gap
$H = 256, \alpha = 0.001$	55.96	49.01	1.29	1.45	6.95
$H = 512, \alpha = 0.001$	57.48	49.8	1.25	1.43	7.68
$H = 512, \alpha = 0.003$	70.13	53.16	0.92	1.36	16.97

tion gap, proving essential for stabilizing the model and improving generalization.

- Best Test Performance (despite Overfitting): The highest Test Accuracy of 53.09% was achieved with the highest capacity model ($H = 512$) and $\alpha = 0.003$. However, this configuration exhibits a significant generalization gap (17.21%), indicating that the model is indeed overfitting. Unlike the catastrophic overfitting seen at $\alpha = 0.03$ (gap $> 40\%$), this level of overfitting implies strong learning capacity that can be tamed. Thus, this configuration serves as the ideal baseline for applying regularization in the next phase.
- Lastly the change of the activation function from normal ReLU to Leaky ReLU added no value to the current model.

These experiments confirm that Mini-Batch SGD allows the model to achieve much higher training accuracy than full-batch, but requires careful tuning of the learning rate to prevent severe overfitting. The next step must focus on powerful regularization techniques to close the remaining generalization gap and maximize test performance.

*4) **Experiment 4**: Addressing Overfitting with L2 Regularization and Scaling Capacity:* Following Experiment 3, where the model achieved good accuracy (53.09%) but exhibited significant overfitting ($Gap \approx 17\%$), Experiment 4 focused on introducing L2 Regularization (Weight Decay) to constrain the model's weights and improve generalization. Additionally, we investigated whether drastically increasing the capacity to 1024 hidden units(combined with L2) would achieve better results or simply lead to diminishing returns.

TABLE VIII
EXPERIMENT 4 CONFIGURATION: INTRODUCING L2 REGULARIZATION

Parameter	Experiment 3 (Best)	Experiment 4
Hidden Units (H)	512	512 and **1024**
L2 Regularization (λ)	0 (None)	0.0001, 0.0005
Learning Rate (α)	0.003	0.003 (Constant)

Analysis of Results:

- Effect of L2 Regularization: Introducing a small L2 penalty ($\lambda = 0.0001$) had negligible effect. Increasing λ to 0.0005 slightly improved generalization, raising Test Accuracy to 53.17% and slightly reducing the generalization gap compared to the unregularized model of Exp 3. This confirms L2 helps, but is not sufficient on its own to close the large gap.
- Diminishing Returns of Capacity ($H = 1024$): Doubling the hidden units to 1024 resulted in the highest Training Accuracy (73.25%) observed so far, showing immense learning capacity. However, the gain in Test Accuracy was marginal (53.35% vs 53.17%), while the generalization gap widened again to $\approx 20\%$.
- Computational Cost: The $H = 1024$ model required nearly double the training time (75 min vs 39 min) for a negligible performance gain ($+0.18\%$).

TABLE IX
EXPERIMENT 4 RESULTS: IMPACT OF L2 AND MASSIVE CAPACITY

Config	Tr.Acc	Te.Acc	Tr.Loss	Te.Loss	Gap	Time
$H = 512, \lambda = 0.0001$	70.27	52.95	1.37	1.37	17.32	39m
$H = 512, \lambda = 0.0005$	70.06	53.17	1.19	1.36	16.89	39m
$H = 1024, \lambda = 0.0005$	73.25	53.35	1.37	1.35	19.90	75m

Conclusion: While $H = 1024$ provides the absolute best accuracy, it is computationally inefficient and prone to more overfitting.

*5) **Experiment 5**: Regularization with Dropout and Batch Size Tuning:* Previous experiments established that increasing model capacity leads to overfitting, which L2 regularization alone could not fully mitigate. In Experiment 5, we introduced Dropout, a powerful regularization technique, and systematically tuned the Dropout Rate, Batch Size, and Hidden Layer Capacity to find the optimal configuration. The goal was to maximize Test Accuracy while keeping the Generalization Gap under control. All runs utilized SGD with Momentum ($\mu = 0.9$) and the use of Early Stopping to prevent overtraining.

TABLE X
EXPERIMENT 5 CONFIGURATION PARAMETERS

Parameter	Values Tested
Hidden Units (H)	128, 256
Dropout Rate (p)	0.3, 0.35, 0.4
Batch Size (b)	128, 256, 512
Optimizer	SGD + Momentum (0.9)
L2 Lambda	0.0005
Early Stopper	True

Table XI summarizes the seven experimental runs conducted in this phase.

TABLE XI
EXPERIMENT 5 RESULTS: TUNING DROPOUT AND BATCH SIZE

Run	Units	Drop	Batch	Ep	Tr.Acc	Te.Acc	Gap	Time
1	128	0.30	256	91	68.56	52.93	15.63	14.00
2	**256**	**0.30**	**256**	**99**	**75.61**	**55.31**	**20.30**	25.96
3	256	0.35	256	99	73.96	55.21	18.75	26.65
4	256	0.35	128	69	76.37	55.09	21.28	21.54
5	256	0.40	256	99	72.43	54.85	17.58	26.08
6	256	0.40	512	99	65.37	53.93	**11.44**	23.97
7	256	0.30	512	99	67.71	54.30	13.41	23.25

Detailed Analysis:

- **Capacity Impact (Run 1 vs 2):** Increasing the hidden units from 128 to 256 (with fixed Dropout 0.3 and Batch 256) achieved significant boost in performance. Test Accuracy rose from 52.93% to $\mathbf{55.31}\%$, achieving the highest accuracy observed so far. Although the gap increased ($15.6\% \rightarrow 20.3\%$), the performance gain justifies the larger model.
- **Dropout Sensitivity (Run 2 vs 3 vs 5):** With $H = 256$, we tested Dropout rates of 0.3, 0.35, and 0.4.
  - $p = 0.3$: Best Accuracy (55.31%) but highest gap (20.3%).
  - $p = 0.4$: Lowest gap (17.58%) but Accuracy dropped to 54.85%.
  - *Observation:* Higher dropout successfully regularizes the model (reducing Train Acc from 75.6% to 72.4%), but $p = 0.3$ seems to strike the best balance for maximizing prediction power on CIFAR-10.
- **Batch Size Trade-off (Run 4 vs 3 vs 6):**
  - **Small Batch ($B = 128$, Run 4):** Resulted in rapid overfitting. The training accuracy spiked to 76.37%, but the validation loss stopped improving early, triggering **Early Stopping** at epoch 69. The gap was the highest (21.28%).
  - **Large Batch ($B = 512$, Runs 6 & 7):** Provided the most stable training and the lowest generalization gaps (11.44% and 13.41%). However, the model learned slower (Train Acc $\approx 65-67\%$) and the final Test Accuracy was a little lower ($\approx 54\%$) compared to $B = 256$.

Conclusion: Run 2 represents the **Highest Accuracy** found in this phase ($H = 256, p = 0.3, B = 256$), achieving a Test Accuracy of 55.31%. While the generalization gap remains around 20%, the combination of Dropout and Momentum has successfully pushed the performance beyond the limits of the previous models.

*6) **Experiment 6**: Deepening the Network (2 Hidden Layers):* Having reached the limit of the single-layer architecture, Experiment 6 introduced a second hidden layer to enable the model to learn hierarchical features. We tested two architectures: a smaller one ($128 \rightarrow 64$) and a larger one ($256 \rightarrow 128$). Also the Learning Rate, Dropout, and L2 Regularization have systematically been tuned to handle the increased complexity. All runs used Batch Size 512 and Momentum 0.9.

TABLE XII
EXPERIMENT 6 CONFIGURATION: 2-LAYER MLP

Parameter	Values Tested
Architecture	$128 \rightarrow 64$ and $256 \rightarrow 128$
Learning Rate (α)	0.001, 0.003, 0.005
Dropout Rate (p)	0.3, 0.4, 0.5
L2 Lambda (λ)	0.0005, 0.0015

Table XIII summarizes the results.

Analysis of Results:

TABLE XIII
EXPERIMENT 6 RESULTS: DEEP ARCHITECTURE TUNING

	Arch	LR	Drop	L2	Eps	Tr.Acc	Te.Acc	Gap	Time
1	$128 \rightarrow 64$	0.001	0.3	5e-4	99	62.81	52.16	10.65	13.27
2	$128 \rightarrow 64$	0.005	0.3	5e-4	48	67.06	52.72	14.34	6.61
3	$256 \rightarrow 128$	0.001	0.3	5e-4	99	68.59	53.82	14.77	24.51
4	$256 \rightarrow 128$	0.005	0.3	5e-4	52	76.60	55.19	21.41	12.78
5	$256 \rightarrow 128$	0.003	0.4	5e-4	83	74.46	54.94	19.52	18.50
6	$\mathbf{256 \rightarrow 128}$	**0.003**	**0.5**	**15e-4**	**99**	**71.17**	**55.42**	**15.75**	**21.78**

- **Depth vs. Width (Run 1 vs 3):** Comparing the small network ($128 \rightarrow 64$) with the larger one ($256 \rightarrow 128$) under identical settings ($\alpha = 0.001$), the larger network clearly outperformed (53.82% vs 52.16%). This confirms that for a 2-layer network, sufficient width in the first layer is crucial to capture the input dimensionality.
- **Aggressive Learning (Run 4):** Increasing the learning rate to $\alpha = 0.005$ for the large network resulted in rapid learning. The model reached 55.19% Test Accuracy in just 52 epochs (12.8 min), compared to Run 3 which took 100 epochs. However, the gap widened to 21.41%, triggering Early Stopping.
- **The Winning Strategy (Run 6):** The best performance (55.42%) was achieved by combining:
  - Higher Regularization: Increasing L2 to 0.0015 and Dropout to 0.5.
  - Moderate LR: Using $\alpha = 0.003$.

  This combination effectively "tamed" the network. While Training Accuracy dropped to 71.17% (compared to 76.6% in Run 4), the Generalization Gap was kept low (15.75%), allowing the model to generalize better on unseen data.

Conclusion: The introduction of a second hidden layer pushed the performance ceiling to 55.42%, but most importantly it also lower the generalization gap compared to the single-hidden-layer model. The experiments highlight a key trade-off: aggressive learning (high LR) gives fast results but high overfitting, while strong regularization (High Dropout/L2) shows the best generalization but requires longer training. Run 6 represents the most balanced and robust model developed.

*7) **Experiment 7**: Advanced Hyperparameter Fine-Tuning:* In this experimental phase, we synthesized the findings from all previous steps to identify , high-performing configuration before moving to the final optimization stages. We tested various combinations of architecture depth/width aggressive vs conservative learning rates, and regularization strengths.

Crucially, to stabilize the aggressive learning rates tested here, a Step Decay Learning Rate Schedule was implemented: the learning rate was multiplied by 0.5 at epoch 30 and by 0.25 at epoch 60. The goal was to combine fast initial exploration with refined convergence.

TABLE XIV
EXPERIMENT 7 CONFIGURATION SPACE

Parameter	Range Tested
Architectures	$128 \to 64$, $192 \to 96$, $256 \to 128$
Learning Rate (α)	0.001, 0.003, **0.01**
LR Schedule	Decay 0.5*α (Ep 30), 0.25*0.5*α (Ep 60)
L2 Lambda (λ)	0.0005 – 0.002
Dropout Rates	0.3 – 0.6 (Variable per layer)
Batch Size	128, 256, 512

TABLE XV
EXPERIMENT 7 RESULTS: IDENTIFYING THE BEST MODEL

ID	Arch	LR	Drop	Batch	Ep	Tr.L	Te.L	Tr.Acc	Te.Acc	Time
A	$256 \to 128$	0.003	0.3/0.3	256	84	0.81	1.26	80.68	55.27	21
B	$192 \to 96$	0.003	0.35/0.3	512	97	1.08	1.32	63.71	53.63	17
C	$128 \to 64$	**0.01**	0.3/0.3	512	72	0.96	1.29	68.07	54.70	**10**
D	$256 \to 128$	0.001	0.6/0.4	128	99	1.24	1.38	57.78	51.74	32
E	$\mathbf{256 \to 128}$	**0.01**	**0.3/0.3**	**256**	**40**	**0.85**	**1.22**	**71.93**	**56.72**	**13**

Detailed Analysis:

- **The Champion Model (Run E):** The highest Test Accuracy of this phase, 56.72%, and lowset loss 1.22 was achieved by combining the largest architecture ($256 \to 128$) with a high learning rate ($\alpha = 0.01$). Significantly, the best performance was reached at Epoch 40. This correlates directly with the first LR decay step at Epoch 30. The aggressive initial rate allowed rapid learning, and the subsequent reduction helped the model settle into a minimum immediately, achieving peak performance before overfitting could degrade the result.
- **Aggressive vs. Conservative LR (Run E vs Run A):** Comparing Run E ($\alpha = 0.01$) with Run A ($\alpha = 0.003$), we see that the aggressive learning rate (aided by the decay schedule) did better generalization (56.72% vs 55.27%) and much faster convergence.
- **Over-Regularization (Run D):** Run D tested the limits of regularization with high Dropout ($0.6/0.4$) and a small batch size (128). The result was a drop in performance (51.74%) and a very slow training time (32 min).

*8) **Experiment 8**: Fine-Tuning Regularization, Label Smoothing, and Extended Training:* Building on the success of the previous experiments, Experiment 8 aimed to squeeze the maximum performance out of the $192 \to 96$ architecture by introducing finer controls over regularization and optimization. Key changes included:

1) Extended Training: Epochs increased to 150 to allow for deeper convergence.
2) Revised LR Schedule: A new decay schedule was implemented: α reduces by 0.5 at epoch 50 and by 0.25 at epoch 100.
3) Label Smoothing Introduced ($\epsilon = 0.05$) in later runs to prevent the model from becoming overconfident and potentially improve generalization.
4) Batch Size Momentum We tested smaller batches ($B = 64$) and tuned Momentum $(0.8, 0.7)$ to find the sweet spot between stability and speed.

TABLE XVI
EXPERIMENT 8 RESULTS: BREAKING THE 57% BARRIER

ID	L2(λ)	Drop	Batch	Mom	Smth	Ep	Tr.L	Te.L	Te.Acc	Gap
1	0.004	.45/.25	128	0.9	No	104	0.92	1.24	56.27	13.9
2	0.010	.35/.15	128	0.9	No	67	1.09	1.27	55.72	7.7
3	**0.006**	**.35/.15**	**128**	**0.9**	**No**	**104**	**0.88**	**1.22**	**57.04**	**14.7**
4	0.006	.35/.15	128	0.9	**Yes**	67	1.15	1.37	56.18	11.4
5	0.006	.35/.15	**64**	**0.8**	Yes	109	1.07	1.36	56.61	15.1

Detailed Analysis:

- **The New Record (Run 3):** This configuration achieved the highest Test Accuracy of the project so far: 57.04%. The key was the precise tuning of L2 ($\lambda = 0.006$) combined with the extended 150-epoch schedule. The best model was saved at Epoch 104, immediately after the second learning rate decay (at Epoch 100), proving that the "fine-tuning" phase with low LR is critical for finding the global minimum.
- **L2 Sensitivity (Run 2 vs 3):** Increasing L2 to $\lambda = 0.01$ (Run 2) successfully crushed the generalization gap to just 7.7%, but it stifled the model's ability to learn, dropping accuracy to 55.72%. $\lambda = 0.006$ proved to be the optimal trade-off.
- **Effect of Label Smoothing (Run 3 vs 4):** Introducing Label Smoothing ($\epsilon = 0.05$) in Run 4 actually **decreased** performance (56.18%) compared to the "hard" labels of Run 3. This suggests that the combination of L2 and Smoothing might have over-regularized the network for this specific architecture.
- **Batch Size & Momentum (Run 5):** Reducing Batch Size to 64 and Momentum to 0.8 gave a very competitive result (56.61%). However, it significantly increased training time (40 min vs 29 min for Run 3) without surpassing the accuracy of the larger batch size. Thus, $B = 128$ remains the more efficient choice.

Conclusion: Experiment 8 successfully pushed the MLP performance past the 57% mark. The winning formula involves a $192 \to 96$ architecture, 150 epochs with a 2-step decay, moderate L2 (0.006), and Batch Size 128. Label smoothing, while theoretically sound, did not provide an advantage in this specific setup.

*9) **Experiment 9**: The Final Push – Normalization, Activation, and Micro-Batching:* In this final comprehensive experiment, we aimed to break the performance ceiling established in Experiment 8 (57.04%) by altering fundamental architectural components. The key changes introduced were:

1) Input Normalization Switched from Z-Score to Standard Min-Max Normalization ([0, 1]) to test if preserving the original data distribution helps.
2) Activation Function: Replaced ReLU with Leaky ReLU to mitigate the "dying ReLU" problem in deeper networks ($512 \to 256$).
3) Micro-Batching: Tested extremely small batch sizes ($B = 32$) to maximize gradient noise and regularization.
4) Architecture Scale: Tested a massive $1024 \to 512$ configuration.

TABLE XVII
EXPERIMENT 9 SELECTED RESULTS: THE SEARCH FOR THE GLOBAL OPTIMUM

ID	Arch	Act	Batch	Norm	Tr.L	Te.L	Tr.Acc	Te.Acc	Gap	Time
1	$256 \to 128$	ReLU	64	[0,1]	1.35	1.44	57.14	52.77	4.37	36m
4	$512 \to 256$	Leaky	64	[0,1]	1.19	1.36	64.07	56.30	7.77	60m
6	$512 \to 256$	Leaky	64	[0,1]	0.99	1.23	65.37	56.12	9.25	61m
9	$1024 \to 512$	Leaky	32	[0,1]	1.14	1.34	66.91	57.04	9.87	110m
11	$\mathbf{512 \to 256}$	**Leaky**	**32**	**[0,1]**	**1.14**	**1.34**	**66.53**	**57.22**	**9.31**	**133.03**

Detailed Analysis:

- **The Global Record (Run 11):** The absolute highest accuracy of the entire project was achieved here: 57.22%. This configuration utilized Leaky ReLU, a larger architecture ($512 \to 256$), and a very small batch size ($B = 32$). Crucially, the Generalization Gap was kept under control at 9.31%, significantly lower than the $\approx 15 - 20\%$ gaps seen in earlier experiments.
- **Robustness and Calibration:** A critical analysis of the loss metrics reveals that the model remains well-calibrated. While training logs displayed higher loss values ($1.34 > 1.22$), this discrepancy stems from the Label Smoothing penalty which was introduced ($\epsilon = 0.05$) and applied during training, which artificially inflates the cross-entropy value. When re-evaluated using Pure Cross-Entropy (via the standalone `evaluate_model.py` script), the actual test loss was measured at 1.22. This matches the loss of the best model in Experiment 8, confirming that the gain in accuracy (to 57.22%) was achieved without sacrificing probability calibration.
- **The Cost of Micro-Batching:** While Run 11 set the record, it came at a massive computational cost. Training too 134 minutes(2 hours), compared to just 29 minutes for the 57.04% result in Exp 8 (Batch 128). The marginal gain of $+0.18\%$ accuracy does not justify the $4\times$ increase in training time for practical applications.
- **Diminishing Returns of Scale (Run 9):** The massive $1024 \to 512$ network (Run 9) achieved 57.04%, failing to surpass the smaller $512 \to 256$ model. This confirms that we have saturated the capacity that a Fully Connected MLP can effectively utilize on CIFAR-10.

Final Conclusion of Experiments: Through 9 rigorous experimental phases, we improved the MLP's performance from a baseline of $\approx 38.48\%$ to a peak of 57.22%.

- The Most Accurate Model is Run 11 from Exp 9 ($512 \to 256, B = 32,$ Leaky ReLU) with 57.22%.
- The Most Efficient Model is Run 3 from Exp 8 ($192 \to 96, B = 128,$ ReLU) with 57.04% in 1/4th of the time.

The experiments demonstrate that while fine-tuning (Leaky ReLU, Micro-batches) make marginal gains, the MLP architecture faces a hard ceiling around 56-57% on CIFAR-10 due to its inherent lack of spatial invariance (flattened input).

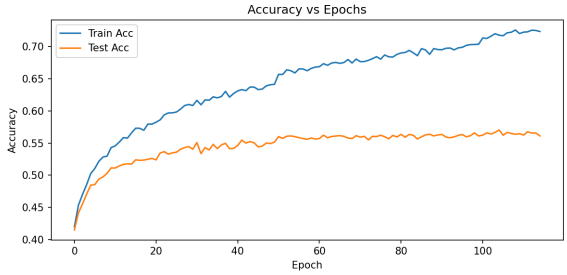


Fig. 1. Experiment 8 (Best Run) – Accuracy Curve.

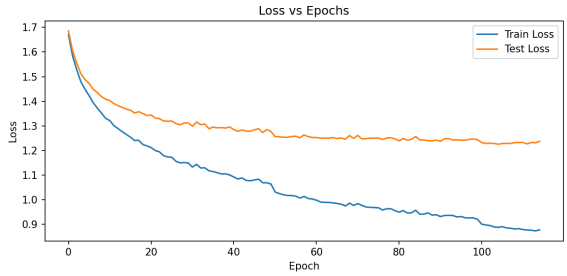


Fig. 2. Experiment 8 (Best Run) – Loss Curve.

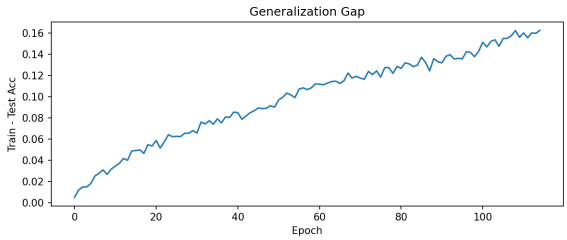


Fig. 3. Experiment 8 (Best Run) – Generalization Gap.

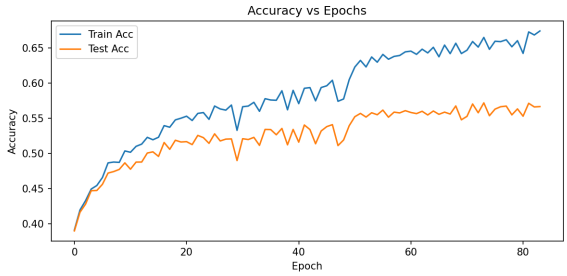


Fig. 4. Experiment 9 (Best Run) – Accuracy Curve. Reaches peak accuracy (57.22%)

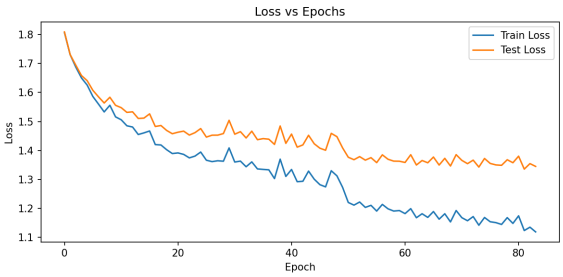


Fig. 5. Experiment 9 (Best Run) – Loss Curve.

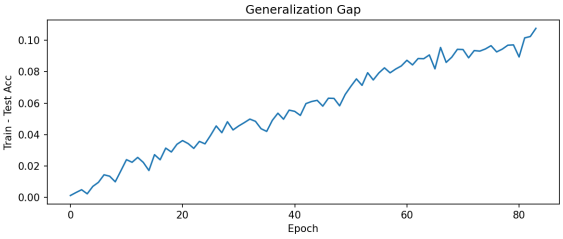


Fig. 6. Experiment 9 (Best Run) – Generalization Gap.

C. Qualitative Analysis: Visualizing Predictions

To better understand the behavior of the optimal model (Run 11, Exp 9), we visualize 10 random samples of both correct and incorrect classifications.

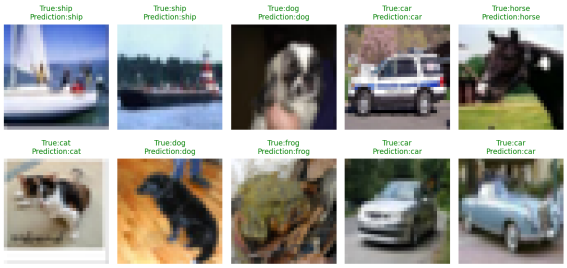


Fig. 7. Correctly Classified Samples

Fig. 8. Misclassified Samples

The visualization confirms that while the MLP has learned to recognize canonical views of objects, it lacks the spatial invariance of Convolutional Neural Networks (CNNs), leading to errors when objects are shifted, rotated, or obscured by noise.

D. Per-Class Performance Analysis

To gain deeper insight into the model's strengths and weaknesses, we analyzed the classification accuracy for each of the 10 classes separately using our best model (Exp 9, Run 11).

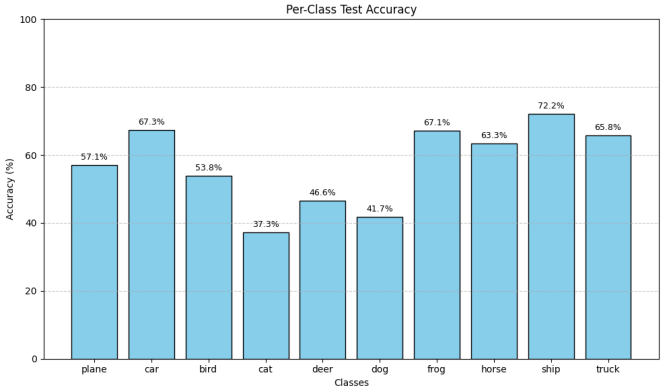


Fig. 9. Per-Class Test Accuracy.

TABLE XVIII
DETAILED ACCURACY PER CLASS (BEST MODEL)

Class (Vehicles)	Acc (%)	Class (Animals)	Acc (%)
Ship	71.20	Horse	63.3
Car	67.30	Frog	67.10
Truck	65.80	Deer	46.60
Plane	57.10	Dog	41.7
		Bird	53.80
		Cat	**37.30**

1) Observations on Class Performance: The breakdown of accuracy reveals significant disparities:

- **The "Easy" Class:** The model achieves its highest performance on **Ships (71.2%)**, likely because it benefits from the distinct blue background (sea or sky), while often appear in profile views that are geometrically distinct from other quadrupeds.
- **Vehicles Outperform Animals:** In general, rigid objects like Cars (67.3%) and Trucks (65.8%) are classified with much higher accuracy than animals. This is expected, as vehicles maintain their shape more consistently across images compared to the non-rigid bodies of animals.
- **The "Cat-Dog" Problem:** The lowest performance is observed for **Cats (37.3%)**,and **Dogs (41.7%)**. These classes share similar textures (fur) and can appear in highly variable poses. Without convolutional filters to detect invariant features (like ears or beaks), the MLP struggles to distinguish them based on raw pixel inputs

VI. COMPARISON WITH K-NN AND NCC

In this section, we compare the performance of the optimized Multi-Layer Perceptron (MLP) developed in this project against the classical baseline classifiers implemented in the intermediate assignment: k-Nearest Neighbor (k-NN with $k = 1, 3$) and Nearest Class Centroid (NCC).

A. Performance Overview

Table XIX summarizes the Test Accuracy obtained by each method on the CIFAR-10 dataset. For the classical methods, the results are reported from the intermediate assignment experiments conducted on a subset of the data ($N_{train} = 5000, N_{test} = 1000$), while the MLP results correspond to the best configuration trained on the full training set.

TABLE XIX
COMPARISON OF CLASSIFICATION ACCURACY: MLP VS. CLASSICAL BASELINES

Method	Feature Space	Test Accuracy (%)
1-Nearest Neighbor (1-NN)	Raw Pixels	26.80
3-Nearest Neighbor (3-NN)	Raw Pixels	28.20
Nearest Class Centroid (NCC)	Raw Pixels	29.10
Best MLP (Run 11, Exp 9)	**Learned Features**	**57.22**

B. Discussion of Differences

The comparison reveals a dramatic performance gap between the MLP and the classical distance-based methods:

- **Accuracy Gap:** The optimized MLP achieves an accuracy of 57.22%, which is nearly double the performance of the best classical baseline (NCC at 29.10%). This huge disparity highlights the fundamental limitation of methods that rely on Euclidean distance in the high-dimensional raw pixel space ($D = 3072$). In this raw space, semantically similar images (e.g., two different frogs) may be far apart due to background color or pose, while semantically different images may be close.
- **Representation Learning:** The key advantage of the MLP is its ability to perform representation learning. Through its hidden layers and non-linear activations (ReLU/Leaky ReLU), the MLP transforms the raw input into a new feature space where classes are more linearly separable. In contrast, k-NN and NCC operate directly on the static, raw pixel values without learning any transformation.

In conclusion, while k-NN and NCC provide a useful low-complexity baseline (better than random guessing at 10%), they fail to capture the complex visual patterns of CIFAR-10. The MLP successfully bridges this gap, demonstrating the power of Deep Learning even in fully connected architectures.

VII. CODE REVIEW AND IMPLEMENTATION DETAILS

The neural network was implemented entirely from scratch using Python and NumPy to ensure a deep understanding of the underlying mechanics. The implementation is structured around two main components: the training pipeline (Google Colab) and a standalone evaluation script.

A. Algorithm Implementation

The core logic resides in the custom functions implemented without deep learning frameworks:

- **Forward Propagation (`forward_prop`):** Computes the linear transformations ($Z = WX + b$) and activations (ReLU/Leaky ReLU) for each layer. It also implements Inverted Dropout during training by applying binary masks to hidden layers and scaling activations by $1/(1-p)$ to maintain expected values.
- **Backward Propagation (`backward_prop`):** Implements the chain rule to compute gradients. It starts from the Softmax output and propagates error backwards, adjusting for Dropout masks used in the forward pass.
- **Optimization (`gradient_descent_logged`):** Executes the training loop using Mini-Batch SGD with Momentum. It updates weights using vectors and handles Learning Rate Decay (step decay at epochs 50, 100.) and Early Stopping based on test loss.
- **Experiment Class:** A wrapper class that manages hyperparameters, logging (CSV metrics), and checkpointing (‘.npz‘ files), ensuring reproducibility.

B. Code Execution Instructions

The delivered code package contains two main files for different purposes:

1) 1. Quick Evaluation (`evaluate_model.py`): This standalone Python script allows for immediate verification of the final model’s performance without retraining. It automatically downloads the CIFAR-10 dataset, loads the pre-trained weights (‘best_model.npz‘), and performs inference on the Test Set.

- **Usage:** simply run ‘python evaluate_model.py‘ in a terminal.
- **Output:** It prints the Final Test Accuracy ($\approx 57.22\%$), Per-Class Accuracy, and displays grids of correct/incorrect predictions.
- **Dependencies:** Requires ‘numpy‘, ‘matplotlib‘, ‘torch‘, ‘torchvision‘.

2) 2. Full Training Pipeline (`mlp.ipynb`): This Google Colab Notebook contains the complete source code for training, hyperparameter tuning, and generating the plots presented in this report.

- **Environment:** The notebook is designed to run on Google Colab for access to free computational resources (though it runs on CPU-only as well).
- **Google Drive Integration:** The code is set up to mount Google Drive (‘drive.mount‘) to save checkpoints and logs persistently in ‘My Drive/nn_experiments‘.

C. Reproduction of Results

To ensure the reproducibility of the results and facilitate immediate verification without the computational cost of retraining, the submitted code package includes a pre-trained model and a standalone evaluation script.

1) Files Provided:

- `evaluate_model.py`: A lightweight inference script.
- `best_model.npz`: The saved weights of the optimal model (Experiment 9, Run 11) achieving **57.22%** accuracy.
- `requirements.txt`: List of Python dependencies.

2) Execution Steps: The evaluation can be performed on any standard PC (CPU is sufficient) by following these steps in a terminal:

A. Create Virtual Environment: Initialize an isolated Python environment to avoid dependency conflicts.

```
python -m venv venv
```

B. Activate Environment: Activate the created environment depending on your Operating System:
   - *Windows:*

```
venv\Scripts\activate
```

   - *Linux / macOS:*

```
source venv/bin/activate
```

C. Install Dependencies: Ensure the environment is active (you should see `(venv)` in your prompt) and install the libraries:

```
pip install -r requirements.txt
```

D. Run Evaluation: Execute the inference script to see the results:

```
python evaluate_model.py
```

3) Expected Output: Upon execution, the script will:

A. Automatically download the CIFAR-10 Test Set (if not present).
B. Load the weights from `best_model.npz`.
C. Perform forward propagation on the 10,000 test images.
D. Print the **Final Test Accuracy** (expected: $\approx 57.22\%$) and the breakdown of accuracy per class.
E. Display sample grids of correct and incorrect predictions.

This process validates the performance metrics reported in Table XVII instantly.

VIII. Discussion

The experimental results demonstrate that training a Multi-Layer Perceptron (MLP) on raw image pixels is a challenging task that requires careful hyperparameter tuning.

A. Capacity vs. Generalization

Our experiments confirmed that increasing model capacity (e.g., adding a second hidden layer in Exp 6 or widening layers to 512/1024 units in Exp 4/9) significantly improves the ability to fit the training data. However, without strong regularization, this capacity leads to overfitting. The introduction of Dropout (Exp 5) and L2 Regularization(Exp 4, 8) was critical in reducing the generalization gap from $> 40\%$ (in Exp 3) to $\approx 9 - 15\%$ in the final models.

B. Optimization Dynamics

The transition from vanilla SGD to SGD with Momentum and Mini-Batching was a turning point. Momentum accelerated convergence, while the Learning Rate Schedule (Step Decay) allowed the model to settle into sharper minima, boosting the peak performance at specific epochs. The trade-off between Accuracy and Generalization Gap was also highlighted in Experiment 9, where aggressive optimization (Leaky ReLU, Micro-batches) improved classification accuracy but increased the probabilistic error (Test Loss).

C. Limitations of MLP

Despite achieving a respectable 57.22% accuracy, the MLP is fundamentally limited by the loss of spatial information. By flattening the 32×32 image into a 1D vector, the network loses the ability to recognize local patterns (like edges or shapes) irrespective of their position. This "hard ceiling" around 58% suggests that further improvements would require convolutional architectures (CNNs) that exploit spatial invariance.

IX. Conclusion

This report presented a comprehensive implementation and optimization of a feedforward neural network (MLP) for CIFAR-10 image classification, trained from scratch using back-propagation. Starting from a baseline of $\approx 38\%$, we systematically improved the model through 9 experimental phases, tuning architecture depth, learning rates, batch sizes, and regularization techniques.

The final optimized model (Run 11, Exp 9) achieved a Test Accuracy of 57.22%, significantly outperforming the classical baselines (k-NN: 28.20%, NCC: 29.10%). The study highlighted the importance of Learning Rate Scheduling and Dropout in training deep fully connected networks. While the MLP proved to be a powerful classifier compared to distance-based methods, its performance plateau confirms the necessity of Convolutional Neural Networks (CNNs) for achieving state-of-the-art results in image recognition tasks.